

Практическая работа №7

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке
<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Новый класс ChartBuilder. Заголовочный файл:

```
1  #ifndef CHARTBUILDER_H
2  #define CHARTBUILDER_H
3
4  #include <QGraphicsScene>
5  #include <QTableWidget>
6
7  class ChartBuilder : public QGraphicsScene
8  {
9  public:
10     explicit ChartBuilder(QTableWidget *sourceTable, QGraphicsView *parent = nullptr);
11     void contextMenuEvent(QGraphicsSceneContextMenuEvent *event);
12     void buildBarChart(const QModelIndexList &selectedCells);
13     void exportToPng(const QString &path);
14
15 private:
16     QTableWidget *m_sourceTable;
17     QMap<QString, double> m_data;
18
19 };
20
21 #endif // CHARTBUILDER_H
```

Вызов функций происходит из контекстного меню TableDocument, там же новый объект добавляется на экран в виде отдельной вкладки m_tabs

```
    emit contentChanged(true);
} else if (selectedAction == actionCreateBarChart && !table->selectedItems().isEmpty()) {
    auto *chart = new ChartBuilder(table);
    chart->buildBarChart(table->selectionModel()->selectedIndexes());
    QGraphicsView *chartView = new QGraphicsView(chart);
    int tabIndex = m_tabs->addTab(chartView, tr("Diagramm"));
    m_tabs->setCurrentIndex(tabIndex);
    emit contentChanged(true);
}
}
```

2) в функцию buildBarCharts передается только выделенная часть таблицы.

Перед началом отрисовки задаются изначальные размеры таблицы, а также значения ширины столбца и отступов между столбцами.

Все элементы перебираются попарно и загружаются в QMap, где первый элемент пары это ключ а второй значение.

Среди полученных пар находится максимальное значение, от которого будут масштабироваться элементы диаграммы.

Далее с помощью QGraphicsLineItem рисуется базовая плоскость диаграммы.

Изначальная позиция по X вычисляется до входа в цикл.

Затем происходит заполнение диаграммы. Для каждой пары создается столбец и подпись, которые добавляются на сцену. Далее позиция инкрементируется для следующего элемента.

```
18 void ChartBuilder::contextMenuEvent(QGraphicsSceneContextMenuEvent *event) {
19     QMenu menu;
20     QAction *actionExportPNG = menu.addAction(tr("Export to PNG"));
21     QAction *actionCloseDiagram = menu.addAction(tr("Close Diagram"));
22     QAction *selectedAction = menu.exec(event->screenPos());
23     if (!selectedAction) return;
24     if(selectedAction == actionExportPNG) {
25         QString path = QFileDialog::getSaveFileName(nullptr, tr("Save Diagram as PNG"), QString(), tr("PNG Files (*.png)"));
26
27         if (!path.isEmpty()) {
28             exportToPng(path);
29         }
30     } else if (selectedAction == actionCloseDiagram) {
31         closeDiagram();
32     }
33 }
34
35 for (auto i = m_data.cbegin(), end = m_data.cend(); i != end; ++i) {
36     qDebug() << qPrintable(i.key()) << ": " << i.value() << "\n";
37     if (i.value() > maxValue) {
38         maxValue = i.value();
39     }
40 }
41
42 qDebug() << "max: " << maxValue;
43
44 QGraphicsRectItem *bar;
45 QGraphicsLineItem *baseLine = new QGraphicsLineItem(-chartWidth / 2, chartMaxHeight, chartWidth / 2, chartMaxHeight);
46 addItem(baseLine);
47 QGraphicsTextItem *caption;
48 double posX = -(chartWidth / 2) + spacer;
49
50 for (auto i = m_data.cbegin(), end = m_data.cend(); i != end; ++i) {
51     qDebug() << "cur X:" << posX;
52     double height = ((chartMaxHeight - 20) / maxValue) * i.value();
53     bar = new QGraphicsRectItem(posX, chartMaxHeight - height, barWidth, height);
54     caption = new QGraphicsTextItem(i.key() + QString("\n(%1)").arg(i.value()), bar);
55     //qDebug() << "capt height: " << caption->boundingRect().height();
56     caption->setPos(posX, chartMaxHeight + spacer);
57     caption->setTextWidth(barWidth);
58     //qDebug() << "capt new height: " << caption->boundingRect().height();
59     // QGraphicsItemGroup *group = createItemGroup({bar, caption});
60     // addItem(group);
61     addItem(bar);
62     addItem(caption);
63     posX = posX + barWidth + spacer;
64 }
65
66 }
```

3) Экспорт изображения. Переопределен метод contextMenuEvent. При выборе действия сохранить в PNG запрашивается путь сохранения, после чего экспортируется сцена

В методе `exportToPng` проверяется, что список не пуст, затем рисуется изображение по размеру сцены и заливается белым фоном. Затем с помощью `painter` отрисовывается сцена и

```

87
88
89 void ChartBuilder::closeDiagram() {
90     clear();
91     m_items.clear();
92     m_data.clear();
93
94     qDebug() << "Diagram closed";
95 }
96
97 void ChartBuilder::exportToPng(const QString &path) {
98     if (m_items.isEmpty()) {
99         QMessageBox::warning(nullptr, tr("Export Error"), tr("No diagram to export"));
100         return;
101     }
102     QRectF sceneRect = this->itemsBoundingRect();
103
104     qreal margin = 50;
105     sceneRect.adjust(-margin, -margin, margin, margin);
106
107     QImage image(sceneRect.size().toSize(), QImage::Format_ARGB32);
108     image.fill(Qt::white);
109
110     QPainter painter(&image);
111     painter.setRenderHint(QPainter::Antialiasing);
112     this->render(&painter, QRectF(), sceneRect);
113     painter.end();
114
115     if (image.save(path, "PNG")) {
116         QMessageBox::information(nullptr, tr("Export Success"),
117                                 tr("Diagram exported successfully to:\n%1").arg(path));
118         qDebug() << "Diagram exported to:" << path;
119     } else {
120         QMessageBox::warning(nullptr, tr("Export Error"),
121                             tr("Failed to save diagram to:\n%1").arg(path));
122         qDebug() << "Failed to export diagram to:" << path;
123     }
124 }
125
126

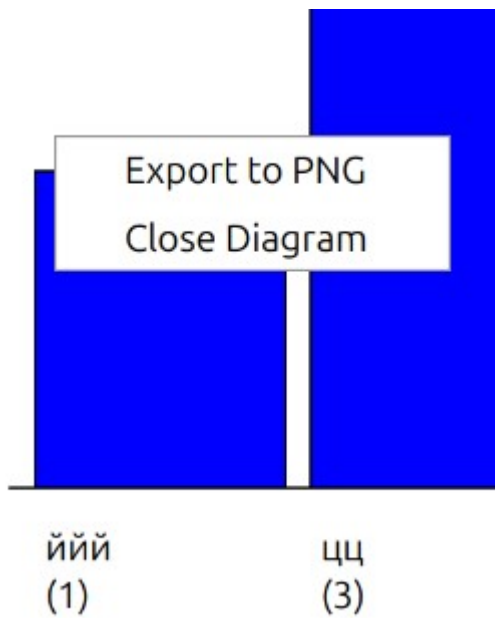
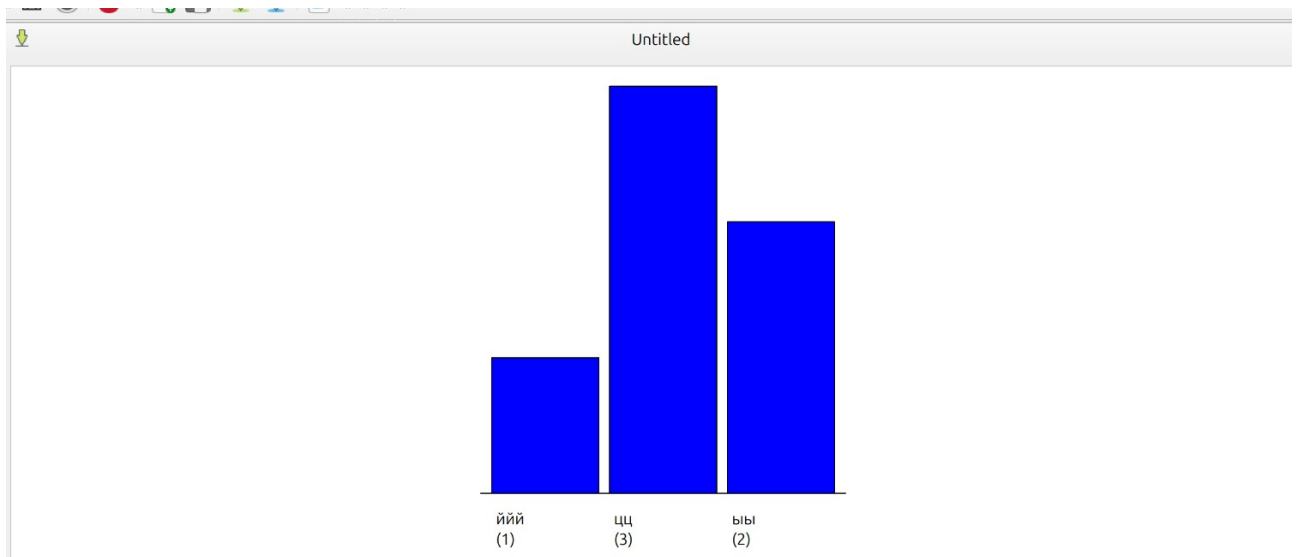
```

изображение сохраняется по указанному пути.

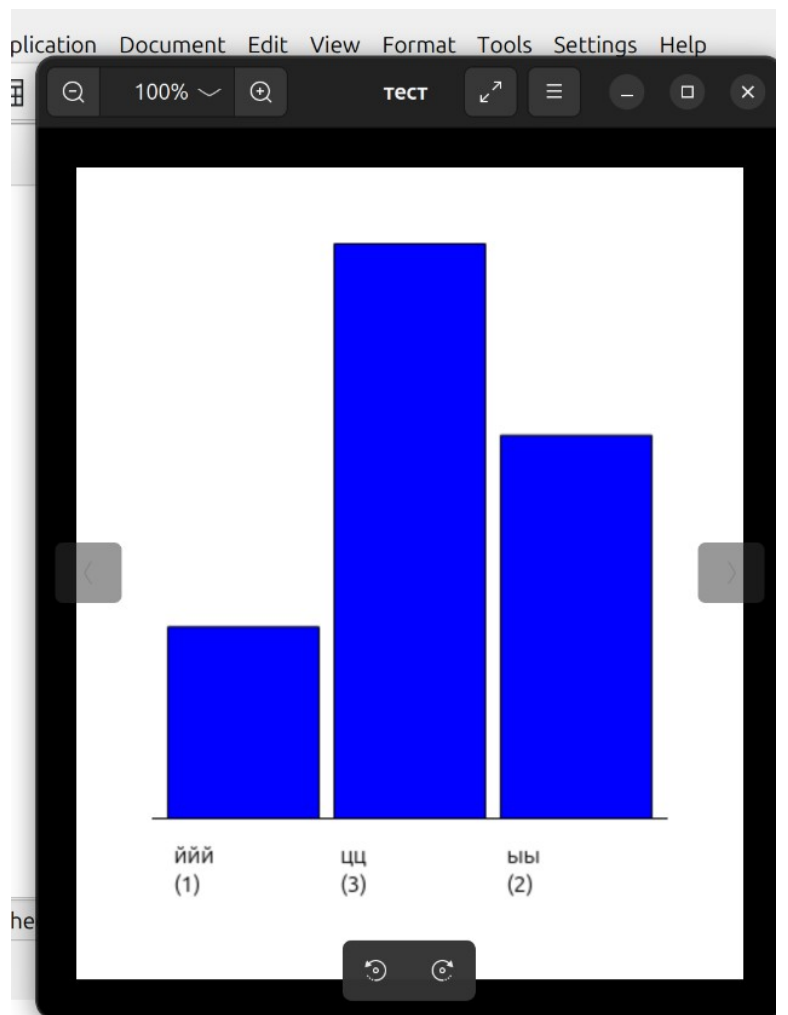
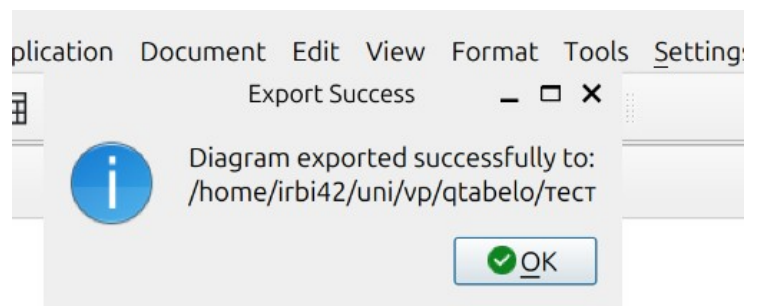
Работа программы:

	Column 1	Column 2	Column 3	Column 4	Column 5
1	ййй	1			
2	ыыы	2			
3	цц	3			
4					
5					
6					
7					
8					

Add Row
 Remove Row
 Add Column
 Remove Column
 Create Bar Chart



Export to PNG
Close Diagram



properties_pages.cpp cur X: -160
recent_document_list.cpp QGraphicsScene::addItem: item has already been added
rename_dialog.cpp cur X: -50
 QGraphicsScene::addItem: item has already been added

Ответы на контрольные вопросы

Как связаны ChartBuilder и QGraphicsView?

ChartBuilder наследуется от QGraphicsScene, а QGraphicsView используется для отображения этой сцены. В коде QGraphicsView передаётся в конструктор (как parent) и служит окном просмотра для построенной диаграммы.

Как сохранить диаграмму в изображение? Какие методы для этого используются?

Вызывается exportToPng(), который создаёт QImage, затем QPainter, и через scene->render(&painter, ...) отрисовывает сцену в изображение. После чего вызывается image.save(path, "PNG").

Как обрабатывается контекстное меню (contextMenuEvent) на сцене?

Переопределён метод contextMenuEvent(QGraphicsSceneContextMenuEvent *event). В нём создаётся QMenu, добавляются действия, меню показывается через exec(event->screenPos()). Выбранное действие выполняется (экспорт или закрытие).

Почему для хранения данных используется QMap<QString, double>?

Для хранения пар "название категории" → "числовое значение". QMap автоматически сортирует по ключу, что удобно для упорядоченного отображения столбцов диаграммы.

Что происходит при многократном вызове buildBarChart с разными данными?

При каждом вызове сначала вызывается clear() (очищает сцену) и m_data.clear(), затем строится новая диаграмма на основе новых выделенных ячеек. Старая диаграмма полностью заменяется.

Как очищается сцена перед построением новой диаграммы?

Вызывается метод clear(), который удаляет все элементы со сцены. Также очищается список m_items и m_data.

Какие классы Qt используются для создания столбцов и подписей?

QGraphicsRectItem — для столбцов диаграммы. QGraphicsTextItem — для подписей (название категории и значение). QGraphicsLineItem — для базовой линии (оси).

Как рассчитывается высота столбцов диаграммы?

$height = ((chartMaxHeight - 20) / maxValue) * \text{значение}$. Где chartMaxHeight = 400 (максимальная высота в пикселях), maxValue — максимальное значение среди данных.

Как обрабатываются очень длинные подписи (перенос, обрезание)?

Устанавливается ширина текста через caption->setTextWidth(barWidth). Текст автоматически переносится на новую строку, если не помещается по ширине.

Как позиционируются столбцы и подписи на сцене?

Сцена центрирована. Столбцы располагаются с шагом: $posX = -(chartWidth / 2) + spacer$, затем увеличиваются на barWidth + spacer. Подписи позиционируются относительно столбца: setPos(posX, chartMaxHeight + spacer).